Cloud Backup Automation using AWS Lambda

Introduction

With the increasing reliance on cloud infrastructure, ensuring data durability and availability has become critical. Manual backup processes are time-consuming, error-prone, and difficult to scale. This project presents an automated backup solution using AWS services that handles the complete lifecycle of backups—from creation to retention and monitoring—without manual intervention.

The system leverages AWS Lambda for serverless execution, EventBridge for scheduling, and other AWS services to ensure a reliable and cost-efficient backup mechanism.

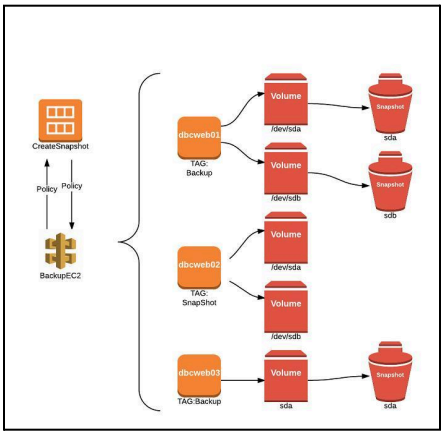
Problem Statement

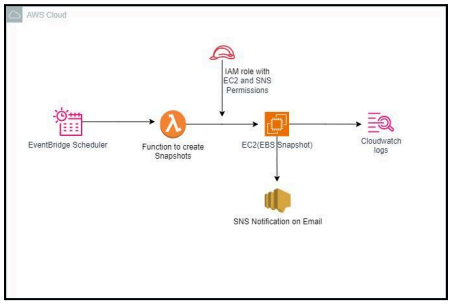
Organizations often face the following challenges:

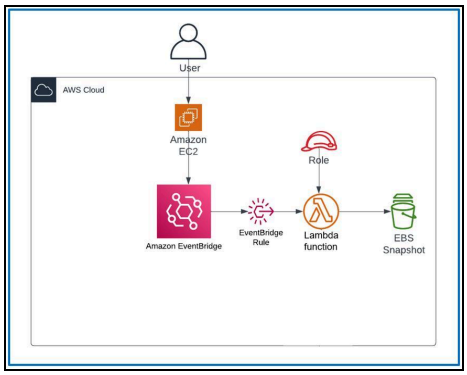
- Manual backup processes leading to inconsistency
- Lack of proper retention policies causing storage overflow
- No centralized monitoring or alerting system
- High operational overhead for managing backups

This project addresses these issues by automating the entire backup workflow.

System Architecture Overview







The architecture follows an event-driven serverless model:

- Amazon EC2 & EBS: Source of data (compute and storage resources)
- AWS Lambda: Executes backup logic
- Amazon EventBridge: Triggers Lambda on a schedule
- Amazon CloudWatch: Logs and monitors execution
- Amazon SNS: Sends notifications
- AWS IAM: Secures access and permissions

Detailed Workflow Explanation

Step 1: Scheduled Trigger

- EventBridge is configured with a cron expression (e.g., daily at midnight)
- It triggers the Lambda function automatically

Step 2: Resource Identification

- Lambda scans EC2/EBS resources
- Uses **tags** to identify which volumes need backup
- This ensures flexibility and avoids hardcoding resources

Step 3: Snapshot Creation

- Lambda creates snapshots of identified EBS volumes
- Each snapshot is tagged with metadata such as:
    - Creation date
    - Instance ID
    - Backup type

Step 4: Retention Policy Enforcement

- Lambda retrieves existing snapshots
- Compares snapshot age with retention policy (e.g., keep last 7 days)

- Deletes older snapshots automatically

Step 5: Monitoring & Logging

- All operations are logged in CloudWatch
- Logs include:
    - Successful snapshot creation
    - Errors or failures
    - Deletion actions

Step 6: Notification System

- SNS sends alerts for:
    - Successful backups
    - Failures or exceptions
- Notifications can be sent via email or SMS

Core Components Explanation

AWS Lambda

AWS Lambda is the backbone of the system. It runs the backup logic without requiring server management.

Responsibilities:

- Identify resources
- Create snapshots
- Apply retention logic
- Send notifications

```python
import boto3
import datetime

ec2 = boto3.client('ec2')
sns = boto3.client('sns')

SNS_TOPIC_ARN = "arn:aws:sns:ap-south-1:account-id:topic-name"
INSTANCE_ID = "i-0123456789abcdef0"
RETENTION_COUNT = 3

def lambda_handler(event, context):
    try:
        instance_details = ec2.describe_instances(InstanceIds=[INSTANCE_ID])
        block_devices = instance_details['Reservations'][0]['Instances'][0]['BlockDeviceMappings']

        snapshots_created = []
        snapshots_deleted = []

        for block in block_devices:
            volume_id = block['Ebs']['VolumeId']

            description = (
                f"AutoSnapshot | Instance: {INSTANCE_ID} | Volume: {volume_id} | "
                f"{datetime.datetime.utcnow().strftime('%Y-%m-%d_%H-%M-%S')}"
            )

            snapshot = ec2.create_snapshot(
                VolumeId=volume_id,
                Description=description
            )

            snapshot_id = snapshot["SnapshotId"]
            snapshots_created.append(snapshot_id)

            ec2.create_tags(
                Resources=[snapshot_id],
                Tags=[
                    {'Key': 'CreatedBy', 'Value': 'LambdaAutoSnapshot'},
                    {'Key': 'InstanceId', 'Value': INSTANCE_ID},
                    {'Key': 'VolumeId', 'Value': volume_id}
                ]
            )
```

```
            lambda_snapshots = ec2.describe_snapshots(
                OwnerIds=['self'],
                Filters=[
                    {'Name': 'volume-id', 'Values': [volume_id]},
                    {'Name': 'tag:CreatedBy', 'Values': ['LambdaAutoSnapshot']}
                ]
            )['Snapshots']

            sorted_snaps = sorted(lambda_snapshots, key=lambda x: x['StartTime'], reverse=True)

            if len(sorted_snaps) > RETENTION_COUNT:
                old_snaps = sorted_snaps[RETENTION_COUNT:]
                for old in old_snaps:
                    ec2.delete_snapshot(SnapshotId=old['SnapshotId'])
                    snapshots_deleted.append(old['SnapshotId'])

        message = (
            f"Snapshot Automation for Instance {INSTANCE_ID}\n\n"
            f"Snapshots Created ({len(snapshots_created)}): {snapshots_created}\n"
            f"Snapshots Deleted ({len(snapshots_deleted)}): {snapshots_deleted}\n"
        )

        sns.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject=f"EC2 Snapshot Report - Instance {INSTANCE_ID}",
            Message=message
        )

        return {"status": "success"}

    except Exception as error:
        sns.publish(
            TopicArn=SNS_TOPIC_ARN,
            Subject=f"Snapshot Automation FAILED - Instance {INSTANCE_ID}",
            Message=str(error)
        )
        raise error
```

Amazon EventBridge

Acts as the scheduler for automation.

Key Role:

- Triggers Lambda at defined intervals using cron jobs
- Enables fully automated backups

Amazon EBS Snapshots

Snapshots are point-in-time backups of EBS volumes.

Advantages:

- Incremental backups (cost-efficient)
- Stored in Amazon S3 (managed by AWS)

Amazon CloudWatch

Provides observability into the system.

Features Used:

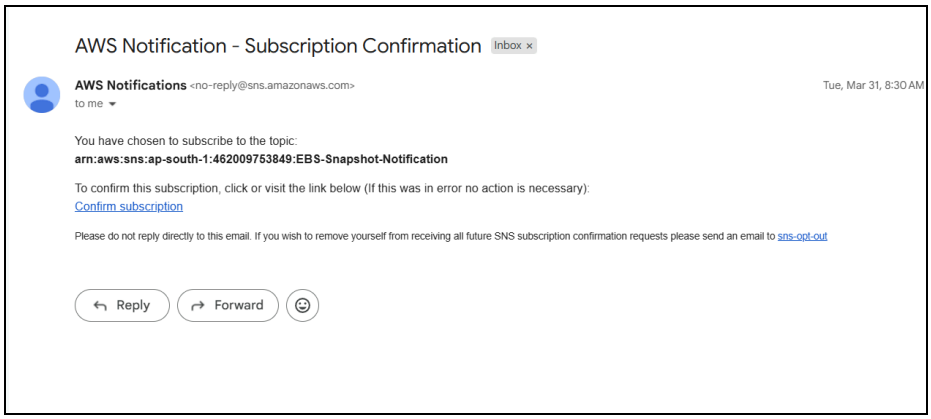
- Logs for debugging
- Metrics for monitoring Lambda performance

Amazon SNS

Ensures real-time alerting.

Use Cases:

- Notify admins of failures
- Confirm successful backup operations



AWS IAM

Controls access securely.

Example Permissions:

- Create/Delete snapshots
- Describe volumes and snapshots
- Publish notifications

Snapshot Retention Strategy

A retention policy is implemented to control storage usage.

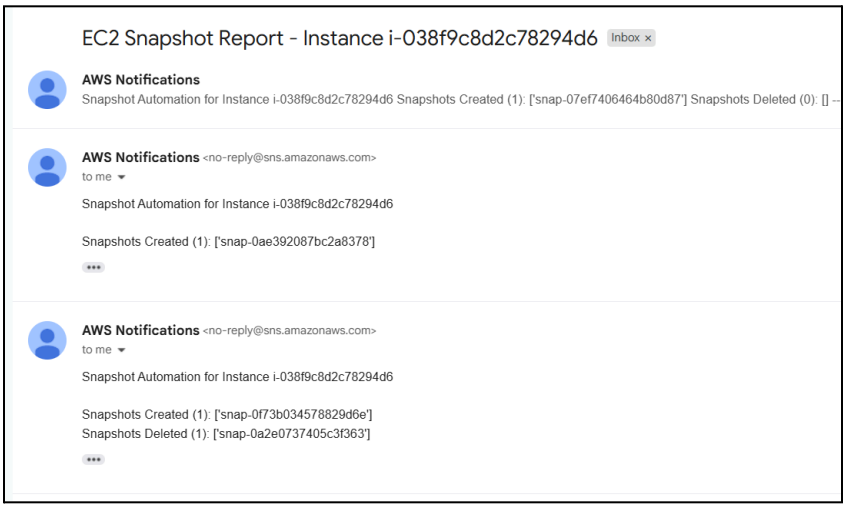
Example Policy:

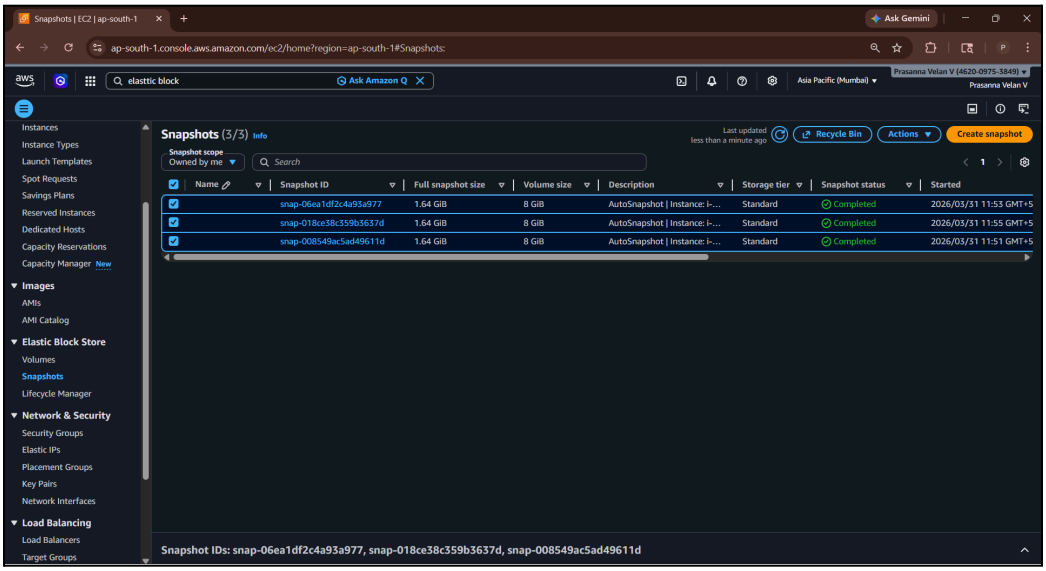
- Keep snapshots for the last **7 days**
- Delete anything older automatically

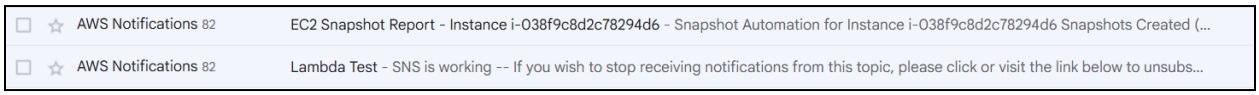
Benefits:

- Prevents storage overflow
- Reduces unnecessary costs
- Maintains only relevant backups

Snapshots Created Automatically and E-mail Received:







Error Handling & Reliability

The system is designed to handle failures gracefully:

- Try-catch blocks in Lambda
- Logging errors in CloudWatch
- Sending failure alerts via SNS

This ensures quick detection and resolution of issues.

Future Enhancements

- Cross-region snapshot replication for disaster recovery
- Integration with AWS Backup service
- Dashboard visualization using CloudWatch dashboards
- Support for RDS and S3 backups
- Automated restore testing

Conclusion

This project demonstrates a robust and scalable approach to cloud backup automation using AWS. By combining serverless computing with event-driven architecture, it eliminates manual intervention while ensuring reliability, cost optimization, and operational efficiency. The solution can be easily extended to support additional AWS services and enterprise-level backup strategies.